

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D039.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 039 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

数据合并 merge/concat

// Joins

KEY CONCEPTS · 三大要点

- ▶ merge
- ▶ concat
- ▶ join

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

选股常常要把两类数据拼起来:行情和财务、价格和因子、股票和行业标签。这一节用真实 A 股的日线加季度财报,带你把 concat 沿轴拼接、merge 按键对齐(类 SQL JOIN)、merge_asof 时间就近对齐这三件套讲透,重点是量化里最实用、也最容易出错的一步——把财报安全地对齐到每个交易日,绝不偷看未来。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解 concat 沿轴拼接:把多只票的数据竖着叠、或多列横着接
- 02 掌握 merge 按键对齐,理解 inner 和 outer join 的区别
- 03 看懂 outer join 产生的 NaN 是"没匹配上"的信号,不是数据本身缺
- 04 会用 merge_asof 把低频财报按时间就近对齐到高频行情,杜绝未来函数
- 05 能把行情和财务拼成一张表,做出一个简单的按 ROE 选股

02 · CONTEXT · 历史与背景

小孙把财报和股价按行号一拼,选股全错

// 不读历史的策略走不远

小孙想做个基本面选股:谁的净资产收益率高就买谁。他下载了股价的日线表和财报的季度表,想当然地把两张表按行号直接拼在一起。结果灾难发生了:股价是每个交易日一行,一年两百多行;财报是每季一行,一年才四行。按行号硬拼,等于让3月1号的股价去配第三季度的财报,时间完全错位。更隐蔽的是,他用了当年最新的年报数据去回测年初的行情,相当于拿着还没公布的财报去买年初的股票,这就是典型的偷看未来,回测好得离谱,实盘一塌糊涂。这一节我们就用 `merge_asof`,教你把每个交易日只对齐到那天之前最近一期已经披露的财报,既拼得上,又绝不偷看未来。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

concat 沿轴拼接

就好比把几张同样格式的表上下摆起来(竖着叠,增加行),或者左右并起来(横着接,增加列)。把十只票各自的日线竖着叠成一张大长表,就是 concat。

CONCEPT_02

merge 按键对齐

等于说就是数据库里的 JOIN:指定一个共同的键(比如股票代码),把两张表按这个键对齐拼到一起。两张 Excel 用学号对齐成绩和身高,就是 merge。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

inner 与 outer join

inner 只保留两张表都有的键,outer 保留并集。outer join 里没对上的地方会变成 NaN,这个 NaN 是"这条没匹配上"的信号,提醒你两张表对不齐,而不是数据本身缺了。

CONCEPT_04

merge_asof 就近对齐

专门给时间序列用的"就近匹配"。它不要求键完全相等,而是为左表每一行找右表里时间最接近的一行。设成 backward,就只匹配当天或之前的记录,天然杜绝偷看未来。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

point-in-time 时点对齐

回测的铁律:某一天只能用那一天真实能看到的信息。财报有披露滞后,merge_asof backward 保证每个交易日只对齐到已经公布的财报,这叫时点正确。

04 · PYTHON LAB · 真实可跑代码

数据合并五件套 — concat 拼接 / merge 类 JOIN / outer 的 NaN / merge_asof 时序就近对齐 / 行情+财务选股

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_039_merge.py - 数据合并:merge(类SQL JOIN)/ concat 拼接 / merge_asof 时序就近对
齐 / outer join 的NaN
# 把行情(日线)和财务(季度ROE/EPS)对齐起来选股,讲透最容易出错的"日期对不上"
# 数据:baostock 日线 + 季频财务(免费、稳定、国内零翻墙)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

pd.set_option('display.width', 160)
pd.set_option('display.max_columns', 30)

STOCKS = {'sh.600519': '茅台', 'sh.600036': '招商银行', 'sz.000858': '五粮液'}
START, END = '2023-01-01', '2024-12-31'

print('==== 0. 数据拉取(baostock 日线 + 季度财务)====')
lg = bs.login()
if lg.error_code != '0':
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')

# 0a 日线收盘
price_frames = {}
for code, name in STOCKS.items():
    rs = bs.query_history_k_data_plus(code, 'date,close', start_date=START,
    end_date=END,
    frequency='d', adjustflag='2')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/5)

数据合并五件套 — concat 拼接 / merge 类 JOIN / outer 的 NaN / merge_asof 时序就近对齐 / 行情+财务选股

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
rows = []
while rs.error_code == '0' and rs.next():
    rows.append(rs.get_row_data())
s = pd.DataFrame(rows, columns=['date', 'close'])
s['date'] = pd.to_datetime(s['date']); s['close'] = s['close'].astype(float)
price_frames[name] = s.assign(股票=name)

# 0b 季度盈利能力(ROE / EPS),baostock query_profit_data 按年+季度
prof_rows = []
for code, name in STOCKS.items():
    for year in (2023, 2024):
        for q in (1, 2, 3, 4):
            rs = bs.query_profit_data(code=code, year=year, quarter=q)
            while rs.error_code == '0' and rs.next():
                r = rs.get_row_data()
                prof_rows.append(r + [name])
            cols = None
# 取一次字段名
rs0 = bs.query_profit_data(code='sh.600519', year=2023, quarter=1)
prof_cols = rs0.fields + ['股票']
bs.logout()

prof = pd.DataFrame(prof_rows, columns=prof_cols)
prof['财报日'] = pd.to_datetime(prof['statDate']) # 财报截止日
prof['ROE'] = pd.to_numeric(prof['roeAvg'], errors='coerce') * 100
prof['EPS'] = pd.to_numeric(prof['epsTTM'], errors='coerce')
prof = prof[['股票', '财报日', 'ROE', 'EPS']].dropna().sort_values(['股票', '财报日'])
print(f'日线合计 {sum(len(v) for v in price_frames.values())} 行,季度财务 {len(prof)} 行')

# ===== 1. concat:沿轴拼接(把多只票的日线竖着叠起来)=====
```


04 · PYTHON LAB · 真实可跑代码 · 续 (3/5)

数据合并五件套 — concat 拼接 / merge 类 JOIN / outer 的 NaN / merge_asof 时序就近对齐 / 行情+财务选股

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('\n==== 1. concat 沿轴拼接 ====')
all_prices = pd.concat(price_frames.values(), axis=0, ignore_index=True)
print(f'concat 把 {len(STOCKS)} 只票的日线竖着叠成一张长表:{all_prices.shape} (行=股票×日期)')
print('每只票行数:', all_prices['股票'].value_counts().to_dict())

# ===== 2. merge:类 SQL JOIN,按键对齐 =====
print('\n==== 2. merge 按键对齐(类 SQL JOIN)====')
# 造一张"行业 + 是否沪深300成分"的小维表,用 merge 贴到行情上
meta = pd.DataFrame({'股票': ['茅台', '招商银行', '五粮液'],
                     '行业': ['白酒', '银行', '白酒'],
                     '入选300': ['是', '是', '是']})
last_price = all_prices.groupby('股票')['close'].last().reset_index()
merged = last_price.merge(meta, on='股票', how='left') # 按股票 这个键左连接
print('最新收盘价 merge 行业维表(how=left,on=股票):')
print(merged.to_string(index=False))

# ===== 3. outer join 的 NaN =====
print('\n==== 3. outer join 会暴露不匹配(产生 NaN)====')
meta_missing = meta[meta['股票'] != '五粮液'] # 故意缺五粮液
inner = last_price.merge(meta_missing, on='股票', how='inner')
outer = last_price.merge(meta_missing, on='股票', how='outer')
print(f'inner join(只留两边都有的):{len(inner)} 行,丢掉了维表里没有的五粮液')
print(f'outer join(两边并集):{len(outer)} 行,五粮液的行业/入选300 变成 NaN — 暴露出谁没匹配上')
print('教训:join 前先想清楚要 inner 还是 outer;outer 的 NaN 是"没对上"的信号,不是数据本身缺')

# ===== 4. merge_asof:时间序列就近对齐(行情 + 财务)=====
print('\n==== 4. merge_asof 时序就近对齐(point-in-time)====')
```

```
# 对每只票:把每个交易日,对齐到"那天能看到的最近一期已披露财报"
out_frames = []
for name in STOCKS.values():
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/5)

数据合并五件套 — concat 拼接 / merge 类 JOIN / outer 的 NaN / merge_asof 时序就近对齐 / 行情+财务选股

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
px = price_frames[name][['date', 'close']].sort_values('date')
pf = prof[prof['股票'] == name][['财报日', 'ROE', 'EPS']].sort_values('财报日')
m = pd.merge_asof(px, pf, left_on='date', right_on='财报日', direction='backward')
m['股票'] = name
out_frames.append(m)
panel = pd.concat(out_frames, ignore_index=True)
print('merge_asof(direction=backward):每个交易日只对齐到"当天之前"最近披露的财报,杜绝未来函数')
sample = panel[panel['股票'] == '茅台'][['date', 'close', '财报日', 'ROE', 'EPS']]
print('茅台抽样(注意 ROE 随财报日跳变,且财报日 ≤ 交易日):')
print(sample.iloc[[0, 60, 120, 200, 350, -1]].to_string(index=False))

# 用对齐好的 ROE 做个简单选股:每天选 ROE 最高的票
panel_valid = panel.dropna(subset=['ROE'])
pick = panel_valid.loc[panel_valid.groupby('date')['ROE'].idxmax()]
pick_count = pick['股票'].value_counts()
print('\n按"当日可见最新 ROE 最高"选股,各票被选中天数:')
for name, c in pick_count.items():
    print(f' {name:8s} {c} 天')

# ===== 画图 =====
plt.rcParams['axes.unicode_minus'] = False
# 图1:concat 后各票净值
fig, ax = plt.subplots(figsize=(11, 5.5))
for name in STOCKS.values():
    s = price_frames[name].set_index('date')['close']
    ax.plot(s.index, s / s.iloc[0], label=name, linewidth=1.6)
    ax.axhline(1.0, color='gray', ls='--', lw=0.8)
    ax.set_title('concat 后三只票净值(起点 1.0)'); ax.legend()
plt.tight_layout(); plt.savefig('chart_01.png', dpi=110); plt.close()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/5)

数据合并五件套 — concat 拼接 / merge 类 JOIN / outer 的 NaN / merge_asof 时序就近对齐 / 行情+财务选股

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 图2:茅台 价格 vs 对齐后的 ROE(阶梯)
fig, ax1 = plt.subplots(figsize=(11, 5.5))
mt = panel[panel['股票'] == '茅台'].set_index('date')
ax1.plot(mt.index, mt['close'], color='#5e81ac', label='收盘价')
ax1.set_ylabel('收盘价', color='#5e81ac')
ax2 = ax1.twinx()
ax2.step(mt.index, mt['ROE'], color='#bf616a', where='post', label='对齐后 ROE(%)')
ax2.set_ylabel('ROE %', color='#bf616a')
ax1.set_title('merge_asof:茅台日线价格叠加按时间就近对齐的季度 ROE')
plt.tight_layout(); plt.savefig('chart_02.png', dpi=110); plt.close()

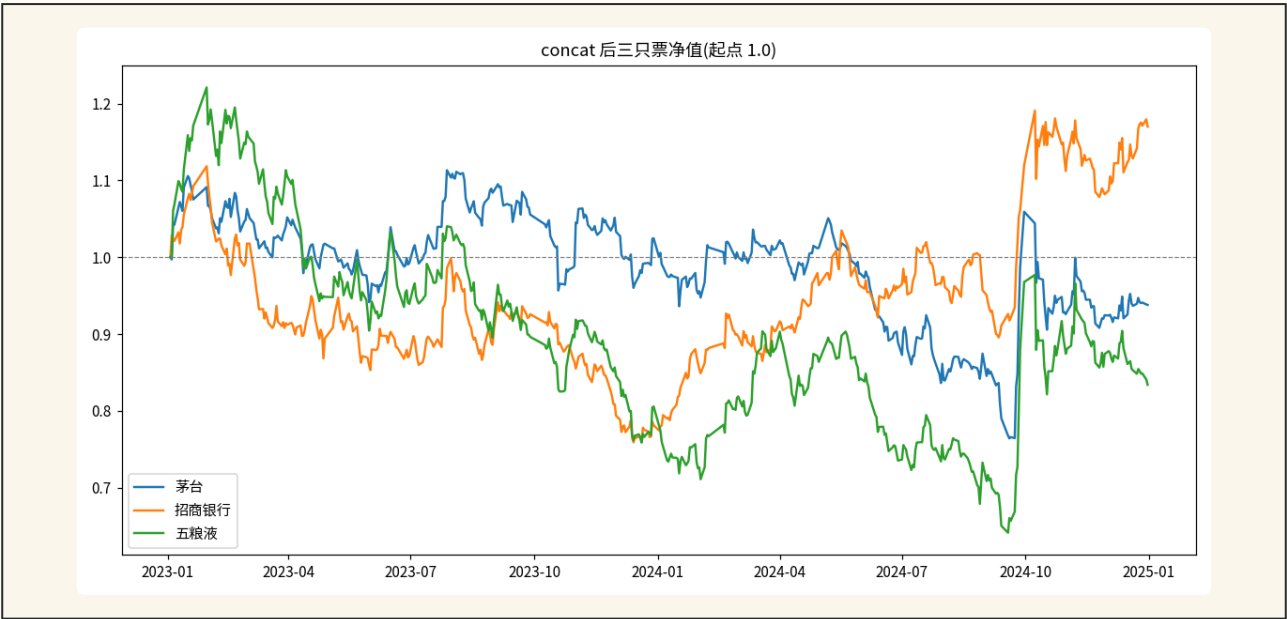
# 图3:三只票 ROE 阶梯对比
fig, ax = plt.subplots(figsize=(11, 5))
for name in STOCKS.values():
    pf = prof[prof['股票'] == name].sort_values('财报日')
    ax.step(pf['财报日'], pf['ROE'], where='post', marker='o', label=name)
ax.set_title('三只票季度 ROE(%)对比'); ax.set_ylabel('ROE %'); ax.legend()
plt.tight_layout(); plt.savefig('chart_03.png', dpi=110); plt.close()

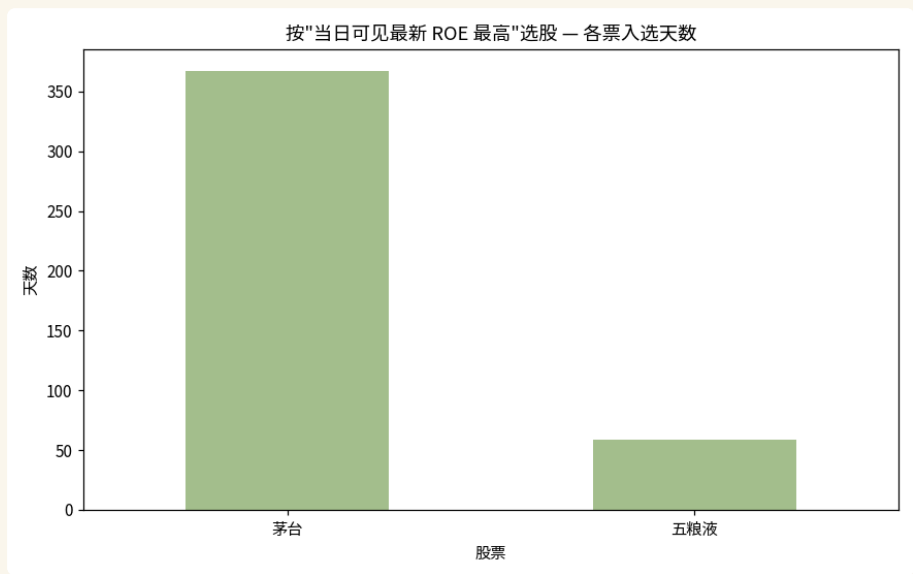
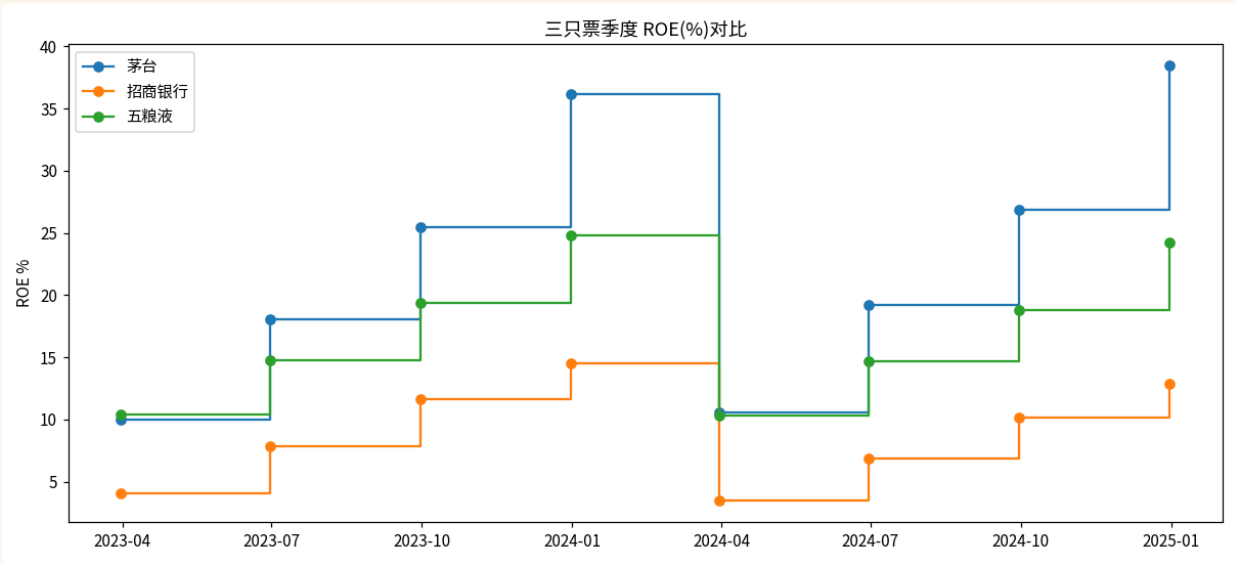
# 图4:选股结果
fig, ax = plt.subplots(figsize=(8, 5))
pick_count.plot(kind='bar', ax=ax, color='#a3be8c')
ax.set_title('按"当日可见最新 ROE 最高"选股 - 各票入选天数'); ax.set_ylabel('天数')
plt.xticks(rotation=0)
plt.tight_layout(); plt.savefig('chart_04.png', dpi=110); plt.close()

print('\n[done] 4 张图已保存,output.txt 见上方全部打印')
```

回测结果

// · matplotlib 真实输出





concat 沿轴拼接

把茅台/招商银行/五粮液 3只票的日线竖着叠成一张长表,共 1452 行(每只 484 天 $\times$ 3),行=股票 $\times$ 日期

merge 按键对齐

最新收盘价按"股票"这个键左连接行业维表:茅台 1469.68/白酒、招商银行 36.72/银行、五粮液 130.77/白酒

inner vs outer join

维表故意缺五粮液:inner join 只剩 2 行(丢掉五粮液),outer join 保留 3 行(五粮液的行业/入选300变 NaN,暴露没匹配上)

merge_asof 时点对齐

茅台每个交易日只对齐到当天之前最近披露的财报:2023-01-03 还没财报是 NaT,4月起 ROE 依次 10.0 $\rightarrow$ 18.06 $\rightarrow$ 25.46,2024-Q1 重置回 10.57,年末 38.43;财报日恒不晚于交易日

ROE 跨年重置

baostock 的 ROE 是年内累计值,每年 Q1 会从低位重新累加,所以 2023-09 的 25.46 到 2024-03 的 10.57 是重置不是暴跌,看财报要看同口径

按可见 ROE 选股

每天选"当日可见最新 ROE 最高"的票:茅台被选中 367 天,五粮液 59 天,招商银行 0 天(ROE 口径下白酒更高)

► 01 **merge_asof 是基本面回测的命门:对齐到"当天能看到"的财报,杜绝偷看未来**

每个交易日只对齐到当天之前最近一期已披露的财报,茅台 2023-01-03 还没出年报所以是空,4月才有 ROE 10.0。这保证了回测里每一天用的都是当时真实能拿到的信息。要是图省事按行号硬拼,或者用全年最终 ROE 去回测年初,就等于拿还没公布的财报去买过去的股票,典型的偷看未来,回测好得离谱、实盘一塌糊涂。

► 02 **outer join 的 NaN 是"没对上"的信号,不是数据缺**

维表故意缺了五粮液,inner join 直接把五粮液丢掉只剩2行,outer join 保留3行但五粮液的行业变成 NaN。这个 NaN 不是数据本身缺,而是在告诉你"这条没匹配上",提醒你两张表对不齐、该去查为什么,而不是当成普通缺失直接填掉。

► 03 **ROE 跨年重置:看财务数据必须看同口径**

茅台 ROE 从 2023-09 的 25.46 跳到 2024-03 的 10.57,看着像暴跌,其实是年内累计口径每年 Q1 归零重算。不懂这个,会把正常的口径切换误读成基本面恶化。这提醒我们,合并财务数据后,一定要搞清楚每个指标的口径和频率,否则对齐了也会读错。

► 04 **concat 竖叠 vs merge 按键:拼表先想清按什么对齐**

concat 把3只票的日线竖着叠成1452行的长表,适合之后 groupby 算横截面;merge 按"股票"这个键把行情和行业维表对齐,类 SQL JOIN。一个按位置叠、一个按键配,用错了要么时间错位、要么张冠李戴,动手前先想清楚你要哪一种。

► 05 **对齐后会选股:茅台 ROE 口径下被选 367 天**

用对齐好的、当日可见的 ROE 做最简单的选股,每天挑 ROE 最高的票,茅台被选中 367 天、五粮液 59 天。这就是一条最朴素的基本面因子策略的雏形,而它能成立的前提,正是 merge_asof 把财报安全地对齐到了每个交易日。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

数据合并

把股价和财报用 merge_asof 对齐,做基本面因子选股,是量化最常见的数据准备步骤

数据合并

多只票的行情用 concat 竖叠成长表,再 groupby 算横截面因子

数据合并

把行情和行业、市值等维表用 merge 贴标签,做行业中性化和分层回测

数据合并

宏观指标(利率、汇率)用 merge_asof 对齐到每日策略信号

数据合并

券商把成交流水按订单号 merge 客户信息,做交易行为分析

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

按行号硬拼不同频率的表

日线和季报行数天差地别,按行号 concat 会让时间完全错位,必须按时间或键对齐。

△ 02

用最新财报回测历史

最致命:拿还没披露的财报去买过去的股票,典型未来函数,回测虚高。用 merge_asof backward 杜绝。

△ 03

merge 后行数暴涨

键有重复时 merge 会产生笛卡尔式的多对多膨胀,合并前先确认键是否唯一。

△ 04

忽略 outer join 的 NaN

outer 后冒出的 NaN 是没匹配上的信号,要去查为什么对不上,而不是直接填掉。

△ 05

merge_asof 忘了先排序

merge_asof 要求两张表都按对齐键排好序,没排序结果会错乱。

数据合并实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 拼表先想清按什么对齐 — 按键用 merge,按位置叠用 concat,按时间就近用 merge_asof。
- 02 财报对行情必用 merge_asof backward — 只对齐当天之前已披露的,杜绝未来函数。
- 03 merge 前确认键唯一 — 防止多对多膨胀导致行数暴涨。
- 04 看懂 outer 的 NaN — NaN 是没匹配上的信号,先查原因再决定处理。
- 05 merge_asof 前先排序 — 两张表都按对齐键排好序。
- 06 concat 注意索引 — 竖叠时常需 ignore_index 重排,避免重复索引。
- 07 对齐后核对样本 — 抽几行看财报日是否都不晚于交易日,确认时点正确。

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 concat 沿轴拼接:竖着叠加行、横着接加列
- 02 merge 按键对齐,类 SQL JOIN,inner 取交集、outer 取并集
- 03 outer join 的 NaN 是"没匹配上"的信号,不是数据缺失
- 04 merge_asof 给时间序列做就近对齐,backward 只看过去
- 05 财报对齐行情必须时点正确,杜绝用未公布数据偷看未来
- 06 merge 前确认键唯一、merge_asof 前先排序
- 07 对齐后抽样核对财报日不晚于交易日

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 merge 和 concat 的核心区别?(提示:merge 按某个共同的键把两张表对齐拼接(类 SQL JOIN);concat 不看键,只是沿某个方向把表拼起来(竖着叠加行或横着接列)。)

Q02 为什么财报对齐行情必须用 merge_asof backward?(提示:财报有披露滞后,backward 让每个交易日只匹配到当天或之前最近一期已公布的财报,保证时点正确、不会用到未来才披露的数据,杜绝未来函数。)

Q03 outer join 之后出现的 NaN 说明什么?(提示:说明这一行在另一张表里没有匹配的键,是"没对上"的信号,提醒你两张表对不齐,应该去查原因,而不是当成普通缺失值直接填掉。)

Q04 小孙把日线和季报按行号直接拼,为什么错?(提示:日线一年两百多行、季报一年才四行,频率完全不同,按行号拼会让某天股价配错季度的财报,时间彻底错位,还可能用到未披露的财报造成未来函数。)

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Wes McKinney 《Python for Data Analysis》(2022, O'Reilly)– merge/concat/join 的权威讲解,含 merge_asof
- ▶ pandas 官方文档《Merge, join, concatenate and compare》(2024)– 各种合并方式的标准参考
- ▶ Marcos López de Prado 《Advances in Financial Machine Learning》(2018, Wiley)– point-in-time 数据与未来函数防范的工业级标准
- ▶ Tulchinsky 等《Finding Alphas》(2019, Wiley)– 因子构建中行情与基本面数据对齐的实务
- ▶ pandas 官方文档《pandas.merge_asof》(2024)– 时间就近对齐的参数与方向说明

NEXT LECTURE · 下一节预告

DAY 040 matplotlib 量化作图

第40 课讲 matplotlib 量化作图:plt 和 ax 两种风格的区别、subplots 多面板布局、twinx 双 y 轴、mdates 日期格式化,带你把净值、回撤、均线信号、成交量画成一张专业的量化分析图。